

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 2 : 머신러닝

DAY4. 머신러닝 결정트리와 회귀트리

목 차

1. 의사 결정 트리		3
2. 회귀 트리와 모델 평가		14
3. 데이터 예측과 회귀 방법		22
4. 로지스틱 회귀 모델		37
5. 역전파		49

1. 의사 결정 트리

의사 결정 트리의 이해 및 분할 정복하기

1. 의사 결정 트리 구조

1) 의사 결정 트리의 이해

- 의사 결정 트리 학습자
 - * 트리를 구조를 활용하여 분류기로 특징과 결과의 관계를 모델링함
 - * 나무의 넓은 몸통부터 시작하여 분할되는 모습 반영
 - * 트리라고 칭함
- 의사 결정 트리 분류기
 - * 예시를 최종 예측 클래스 값 결정으로 분기화 시키는 구조
 - * 루트 노드(root node)에서 시작해서 결정 노드(decision node) 통과
 - * 결정 결과를 나타내는 분기(branches)로 데이터 분할
 - * 분기는 두 개 이상 '예' 또는 '아니오' 로 표시
- 최종 결정 트리
 - * 터미널 노드(terminal node), 잎 노드(leaf nodes)에서 분류 결정(종료)

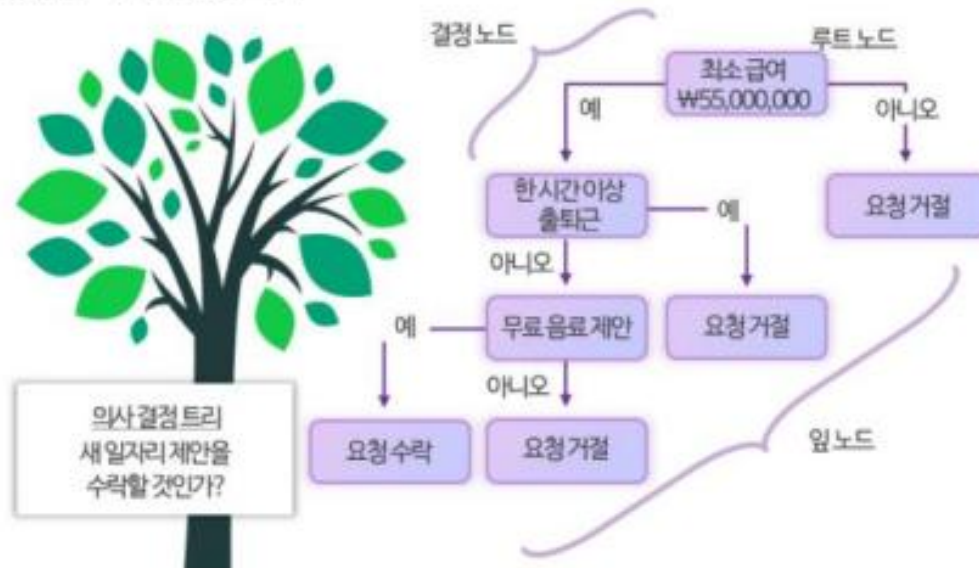
2) 의사 결정 트리의 장점

- 의사 결정 트리 알고리즘
 - * 모델이 생성됨
 - * 사람이 모델 구조를 인식 할 수 있음
- 모델의 작동
 - * 특정 작업에 어떻게 정상적으로 작동하는지 여부
 - * 정상적인 작동에 대한 통찰력 제공
- 법적인 이유
 - * 분류 방법이 명확한 애플리케이션
 - * 다른 사람과 결과를 공유할 경우 의사 결정 트리가 적합함

의사 결정 트리의 이해 및 분할 정복하기

2) 의사 결정 트리의 장점

◦ 결정 절차와 결정 트리



3) 의사 결정 트리의 용도

- 명확한 문서화에 활용
- 균형 있는 신용 평가 모델 가능
- 경영진 또는 광고 대행사의 고객 행동 마케팅 연구
- 실험 측정, 증상, 질병 진행률과 질병 진단
- 분할 정복

* 의사 결정 트리 : 재귀 분할(recursive partitioning)의 휴리스틱(heuristics) 사용

* 방법

- 분할 정복(divide and conquer)으로 알려짐
- 데이터를 부분 집합으로 나눔
- 더 작은 부분 집합으로 반복해서 나눔

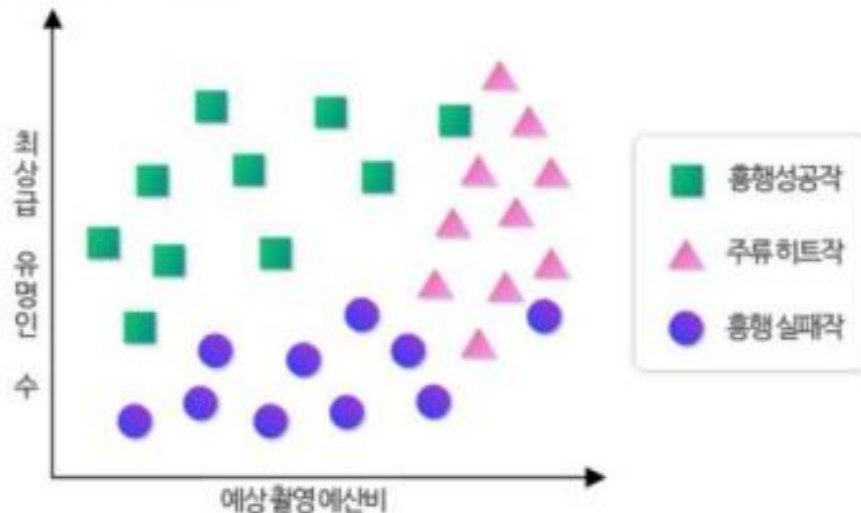
* 알고리즘의 데이터가 동질적이라고 판단함

* 종료 조건 과정이 만족할 때까지 계속함

의사 결정 트리의 이해 및 분할 정복하기

3) 의사 결정 트리의 용도

- 예산 관계를 나타낸 산포도



4) 의사 결정 트리의 장단점

- 장점

- * 일반적으로 문제에 잘 적용되는 범용 분류기
- * 자동화된 학습 과정(숫자 특징, 명목 특징, 누락 데이터를 다룸)
- * 비중이 작은 특징은 제외함
- * 작은 데이터 셋과 큰 데이터 셋에 모두 사용
- * 작은 트리로 해석될 수 있는 모델
- * 다른 복잡한 모델에 비해 더 효율적임

- 단점

- * 의사 결정 트리 모델의 레벨 단계 수가 많은 특징으로 분할
- * 모델의 과적합 또는 과소적합 문제 포함
- * 축 평행 분할에 의존하므로 관계의 모델링이 어려움
- * 훈련 데이터에서의 작은 실험 변화가 큰 결정 로직의 변화 초래
- * 보다 큰 트리는 해석이 어려움
- * 트리 결정은 직관적이지 않음
- * 보완책으로 Random Forest, Boosting, Bagging 등 해결 방안 도입

의사 결정 트리의 이해 및 분할 정복하기

2. 의사 결정 트리 분류

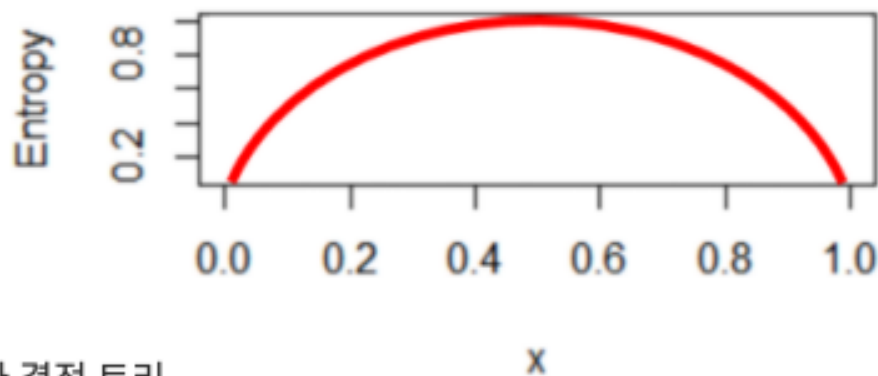
1) 최고의 분할 선택

○ 엔트로피(Entropy)

* 트리 분할 후보 식별 방법

- C5.0 알고리즘은 엔트로피(Entropy)를 사용
- 엔트로피 : 집합에서 무작위성 또는 무질서를 정량화 함
- 엔트로피가 높은 집합은 매우 다양함
- 같은 집합에 속해 있는 서로 다른 아이템들 간에 공통점이 없음
- 정보를 거의 제공하지 않음

* 두 클래스 결과에서 하나의 클래스 비율 현황의 전체 엔트로피



* 의사 결정 트리

- 엔트로피를 감소시키는 분할 검색
- 궁극적으로 그룹 내 동질성을 증가하게 함
- 엔트로피 값이 증가할수록 정보가 증가함
- 엔트로피가 최상위 집합을 루트 노드에 설정
- 정보 획득량이 높을수록 동질적 그룹의 생성에 유리

* 정보 획득량이 '0'인 경우

- 분할하면 엔트로피가 감소하지 않음
- 최대 정보 획득량 : 분할 전 엔트로피와 정보 획득량이 동일함
- 분할 후 엔트로피가 0이라는 것을 의미함
- 분할 구분으로 완전히 동질적인 집합 생성

의사 결정 트리의 이해 및 분할 정복하기

2) 의사 결정 트리 가지치기(pruning)

- 조기 종료(early stopping) 및 사전 가지치기(pre-pruning)
 - * 사전의 특정한 시점에 트리 성장 종료
 - * 특정한 기준을 설정하고, 초반에 마침
 - sample이 10개 이하로 감소하면 가지치기를 멈춤
 - * 트리가 불필요한 일을 하지 않게 하므로 설득력이 약함
 - * 트리가 크게 자랐다면 학습해야 할 핵심적인 패턴을 간과할 수 있음
- 사후 가지치기(post-pruning)
 - * 트리를 의도적으로 크게 한 후 트리 크기를 적절한 수준의 감소로 앞 노드를 자름 → 사전 가지치기보다 효율적인 방법
 - * 사전에 트리를 키우지 않을 경우 의사 결정 트리의 적합한 깊이 판단이 어려움
 - * CART와 C5.0 알고리즘에서 많이 적용

의사 결정 트리를 이용한 위험 요소 식별

1. 의사 결정 트리 식별

1) 의사 결정 트리 위험 식별

- 신용의 유효성 제한

- * 은행의 대출 시스템 강화
- * 위험 대출을 더 정확하게 검색하기 위해 머신러닝으로 전환
- * 의사 결정 트리
 - 정확성이 높은 통계 모델로 표현하는 능력
 - 은행 업계에서 사용
- * 자동화된 신용 평가 모델
 - 신용카드 발송의 자동화와 실시간 온라인 승인 절차 구현
 - 의사 결정 트리를 활용하여 위험한 은행 대출 식별

2) 1단계 : 분석 데이터 수집

- 신용 모델 아이디어

- * 신용 모델 아이디어 : 채무 불이행 위험과 예측 가능한 요소 식별
- * 과거 은행 대출에 비해 대량의 데이터 취급
 - 대출의 채무 불이행 여부, 신청자의 채무 정보 입수, 신용기관의 대출 현황 정보 데이터
- * 신용 데이터 셋
 - 1000개의 대출 예시에 대한 데이터 셋
 - 대출 신청자의 특성으로 구분하는 일련의 숫자 특징
 - 명목 특징을 포함하고 패턴을 결정함
 - 클래스 변수 : 대출의 채무 불이행 여부 판단

의사 결정 트리를 이용한 위험 요소 식별

3) 2단계 : 분석 데이터 탐색과 준비

◦ 데이터 탐색

- * 팩터 변수를 포함한 신용(credit) 데이터프레임 생성
- * 1,000개의 관측과 17개의 특징
- * 특징
 - 팩터 데이터 타입
 - 정수 데이터 타입
- * 채무 불이행 예측(대출 특징을 table()로 출력)
- * 신청자의 수표 계좌
- * 저축 계좌 잔고의 범주형 변수로 저장
- * 랜덤 데이터 훈련과 데이터 셋 구조 생성
- * 의사 결정 트리 생성을 위한 훈련 데이터 셋
- * 새 데이터에 대한 모델 성능 평가
- * 데이터 셋의 데이터를 두 부분으로 분리
- * 훈련 데이터에 80%, 테스트를 위해 20% 사용
- * 200개의 레코드로 신규 신청자 시뮬레이션에 활용

의사 결정 트리를 이용한 위험 요소 식별

4) 3단계 : 분석 데이터 모델 훈련

◦ 데이터 훈련(install.packages("C50") 패키지 설치)

* summery() 함수 호출

- 수표 계좌 잔고를 모르거나, 200DM 이상이면 '채무 불이행 가능성 없음'으로 분류
- 수표 계좌 잔고가 0보다 작고, 1~200DM 이하인 경우
- 대출 이력이 많으면 '채무 이행 가능성 있음'으로 분류

* summery()의 부정확 행렬

- 부정확한 레코드를 표시하는 교차표의 Error 헤딩
- 모델의 오류율은 15.1%
- 800개의 훈련 인스턴스 중 138개를 제외하고 정확하게 분류함
- 총 26개는 no값을 yes로 잘못 분류함(FP 거짓 긍정)
- 95개는 yes값을 no로 잘못 분류함(FN 거짓 부정)

의사 결정 트리를 이용한 위험 요소 식별

2. 의사 결정 트리 식별 평가

1) 4단계 : 분석 모델 성능 평가

- 의사 결정 트리를 테스트 데이터 셋에 적용하려면 predict() 함수를 사용
 - * predict() 함수 : 예측된 클래스 값 벡터 생성
- gmodels 패키지의 CrossTable() 함수 이용 → 벡터와 실제 클래스 값의 차이 비교
- prop.c 와 prop.r 파라미터를 FALSE로 설정
 - * 표에서 열과 행 비율 제거
 - * 나머지 백분율 prop.t는 전체 레코드 개수에 대한 셀의 레코드 비율 표시
- '테스트 집합의 200개 대출 신청 레코드' 분석 모델의 성능 분석
 - * '채무 불이행이 되지 않음' : 120개
 - * '채무 불이행이 되었음' : 25개
 - '채무 불이행이 되었음'을 모델이 정확하게 예측
 - * 모델 예측 정확도 : 73%
 - * 모델 예측 오류율 : 27%
 - 초기 모델이므로 성능이 부족할 수 있음
 - * 62개의 대출 채무 불이행 중 25개(약 40%)만 정확하게 예측됨
 - * 채무 불이행으로 인한 손실 은행의 손해에 따라 크게 발생

의사 결정 트리를 이용한 위험 요소 식별

2) 5단계 : 분석 모델 성능 향상

- 실시간 신용 평가 애플리케이션으로 배포하기에는 모델의 오류율이 높음
- 모든 테스트 케이스에 대해 '채무 불이행 아님'으로 예측한 경우, 67%의 정확도
- 900개의 예시에서 채무 불이행을 예측하는 것은 어려운 문제
- 의사 결정 트리의 정확도 향상
 - * 적응형 부스팅(adaptive boosting) 추가
 - 알고리즘을 개선한 방법 중 하나
 - 여러 개의 의사 결정 트리 생성
 - 각 예시에 대해 최고 클래스를 투표하는 과정
 - 부스팅은 다른 머신러닝 알고리즘에 비해 일반적으로 적용
 - * 장점과 단점이 상호 보완적임
 - 다수의 학습자를 조합하여 분류기의 정확도를 향상시킴
 - * 부스팅에 독립적인 의사 결정 트리의 개수
 - trials 파라미터 추가
 - trials 파라미터의 상한선 설정
- 의사 결정 트리의 정확도 향상
 - * 추가적인 시행의 정확도 향상이 어려움(알고리즘은더이상트리를추가하지않음)
 - * 20회 실행 결과 데이터 오류율이 약 25% 정도로 2%감소하는 표준 상태
- 오류율 부스팅
 - * 모델은 여전히 채무 불이행을 예측하지 못함 (41.9% 예측)
 - * 더 크게 개선되지 않는 이유
 - 작은 훈련 데이터셋과 함수 관계
 - 본질적으로 해결하기 어려운 문제

의사 결정 트리를 이용한 위험 요소 식별

2) 5단계 : 분석 모델 성능 개선

◦ 결과

- 정확도 75%
- 오류율 25%
- 기존 오류율 27%에서 25%로 2% 감소
- 채무 불이행은 $26/62 = \text{약 } 42\%$ 로 예측

actual Repay	predict Repay		Row Total
	no	yes	
no	125 0.625	13 0.065	138
yes	36 0.180	26 0.130	62
Column Total	161	39	200

2. 회귀 트리와 모델 평가

데이터 트리에 회귀 추가하기

1. 회귀 트리 모델

1) 모델 트리 구조의 이해

- 모델 트리 방식

- * 모델 트리 : 회귀 트리와 동일한 방식

- * 잎 노드에서 노드에 도착한 예시로 다중 회귀 모델 구축

- * 잎 노드의 개수에 따라 다중 회귀 모델을 수십 개 구축함

- * 모델 트리가 동등한 회귀 트리보다 이해는 어렵지만 모델이 보다 정확함

2) 트리에 대한 회귀 추가의 장점

- 의사 결정 트리의 장점과 숫자 데이터를 모델링하는 능력

- 사용자가 모델을 사전에 명시하지 않아도 됨

- 자동 특징 선택 사용으로 인해 많은 개수의 특징이 해당 방식에 적용

- 선형 회귀보다 일부 데이터 타입에 잘 맞춤

- 모델의 해석에서 통계 지식은 필요 없음

3) 트리에 대한 회귀 추가의 단점

- 선형 회귀만큼 잘 알려져 있지 않음

- 많은 훈련 데이터가 필요함

- 결과에 대한 개별적인 특징으로 전체적 순영향을 찾아내기 어려움

- 큰 트리 : 회귀 모델보다 해석이 어려울 가능성이 있음

데이터 트리에 회귀 추가하기

2. 숫자 예측 모델

1) 숫자 예측 작업

- 의사 결정 트리
 - * 작업의 특징이 많음
 - * 특징과 결과 사이가 매우 복잡함
- 회귀 모델링
 - * 숫자 데이터의 분포를 가정했을 경우 실제 데이터 본질에 위배됨
- 선형적인 관계에 적합함
- 트리는 적합하지 않음

2) 트리에 대한 회귀 추가의 장점

- 분류용 트리와 동일한 방식
 - * 루트 노드에서 분할 정복 전략으로 데이터 분할
 - * 조건 : 결과의 동질성을 증가시키는 특징으로 분할
 - * 분류 트리는 동질성을 엔트로피로 측정
 - * 숫자 데이터 : 엔트로피가 정의되지 않은 상태
- 숫자 의사 결정 트리
 - * 평균, 분산, 표준 편차, 절대 편차 등
 - * 통계량으로 동질성 측정

회귀 트리와 모델 트리로 품질 평가하기

1. 회귀 트리와 모델 트리 준비

1) 모델 트리로 와인의 품질 평가하기

- 전문가의 와인 평가를 모방할 수 있는 시스템
 - * 회귀 트리와 모델 트리 사용
- 트리 모델은 쉽게 이해하기 위해 와인 양조자가 와인의 긍정적인 평가에 기여한 주요 요인 검색
- 시스템 : 시음할 때 평가자의 기분 또는 미각의 피로 요소로 인한 어려움 여부 이해
- 컴퓨터를 이용한 와인 테스트
 - * 보다 더 객관적이고 일관적임
 - * 공정한 평가 뿐만 아니라 향상된 제품을 만들 수 있음

2) 1단계 : 분석 데이터 수집

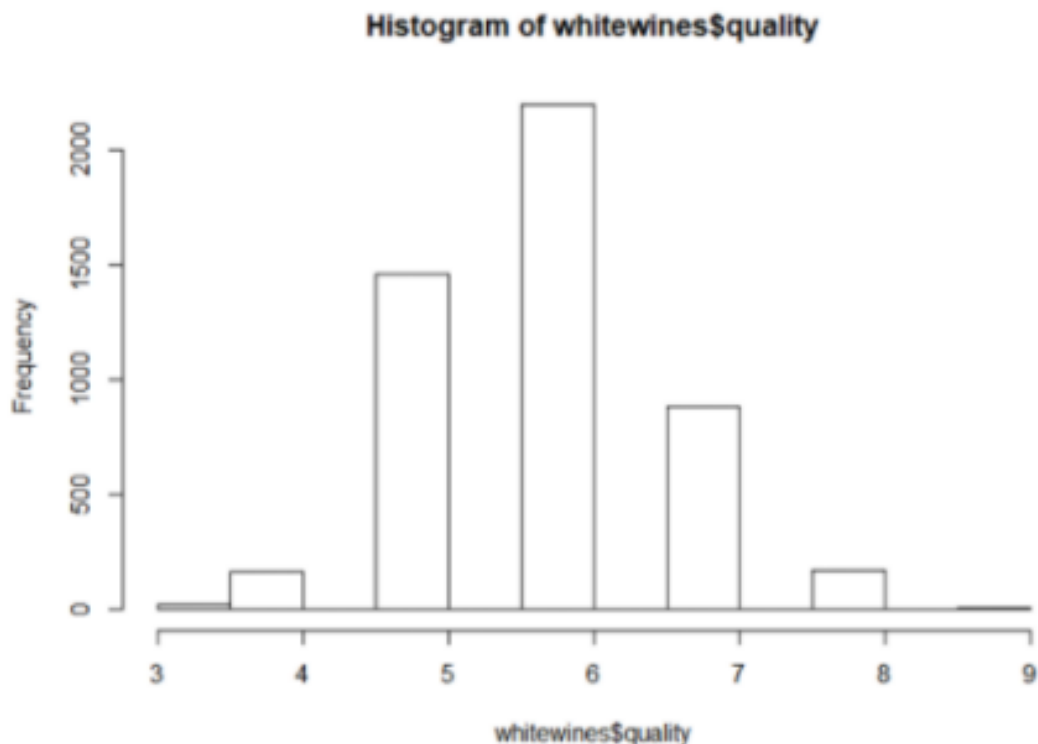
- 와인 평가 모델 개발 데이터
 - * 레드 와인과 화이트 와인의 종류로 와인 평가 정도를 구분함
 - * 기여한 요인에 대한 차이
 - * 화이트 와인만 평가함
 - * 화이트 와인 데이터
 - 4,898개의 와인 샘플과 11가지 화학 속성에 대한 정보
 - 각 와인별 실험실 분석 : 와인의 특성 측정 (산도, 함유량, 염화물, 황, 알코올 등)
 - 평가된 품질 척도의 범위 : 0(매우 나쁨)~10(매우 우수)
 - 평가 기준에 동의하지 않은 심사 위원은 중앙값 사용
 - 회귀 트리과 모델 트리를 이용해 모델을 유지하면서 회귀 결과를 점차 개선함

회귀 트리와 모델 트리로 품질 평가하기

3) 2단계 : 분석 데이터 탐색과 준비

◦ 모델의 성능 평가

- * 적합한 결과 변수의 분포도 조사
- * 와인 사이의 품질 차이가 거의 없음
- * 와인이 매우 좋음, 매우 나쁨 중 하나로 분포됨
- * 모델의 설계 방식에 영향을 줌
- * 심각한 문제 여부의 확인 가능
- * 훈련과 테스트에서 사용 조건에 따라 집합을 75%와 25%로 구분하여 사용
- * 테스트 데이터는 트리 기반 모델로 성능 평가
- * 품질 등급 분포

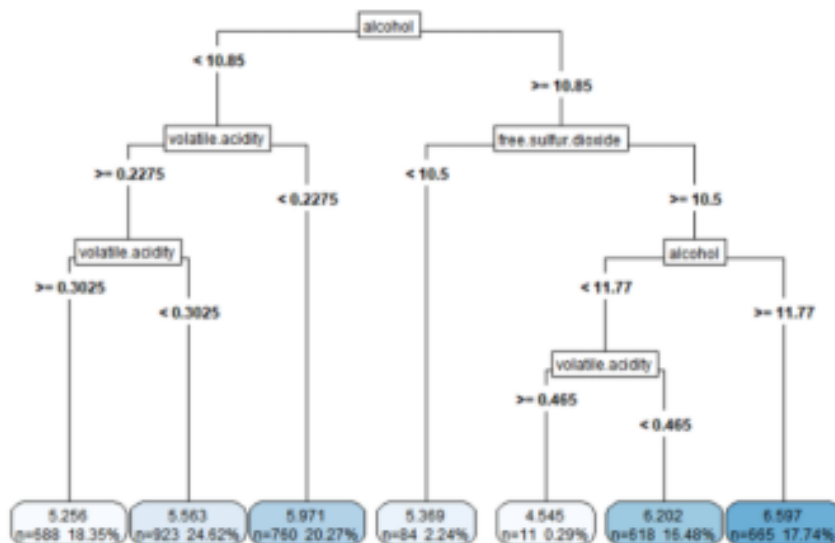


회귀 트리와 모델 트리로 품질 평가하기

4) 3단계 : 분석 데이터 모델 훈련

- 회귀 트리 모델의 시각화
 - * rpart(재귀적 분할) 패키지로 회귀 트리 구현
 - * 문서화와 모델의 시각화
 - * 노드는 터미널 노드 또는 잎 노드로, 노드에서 예측
- w.rpart
 - * 전체 노드 : 3749개
 - * 2371개 : $\text{alcohol} < 10.85$, 1378개 : $\text{alcohol} \geq 10.85$ 예측
 - * 와인의 품질에 가장 중요한 예측 변수를 가장 먼저 사용
 - * $\text{alcohol} < 10.85$, $\text{volatile.acidity} \geq 0.2275$ 인 모든 와인 샘플
 - * quality가 5.43으로 예측됨
- 회귀 트리 모델의 시각화
 - * 의사 결정 트리 시각화(rpart.plot() 함수로 의사 결정 트리 생성)
 - * rpart.plot() 함수 : rpart 모델 객체에서 트리 다이어그램 생성
 - * 기본 의사 결정 트리 다이어그램

```
rpart.plot(w.rpart, digits = 5)
```



회귀 트리와 모델 트리로 품질 평가하기

4) 3단계 : 분석 데이터 모델 훈련

◦ 도식화 분석

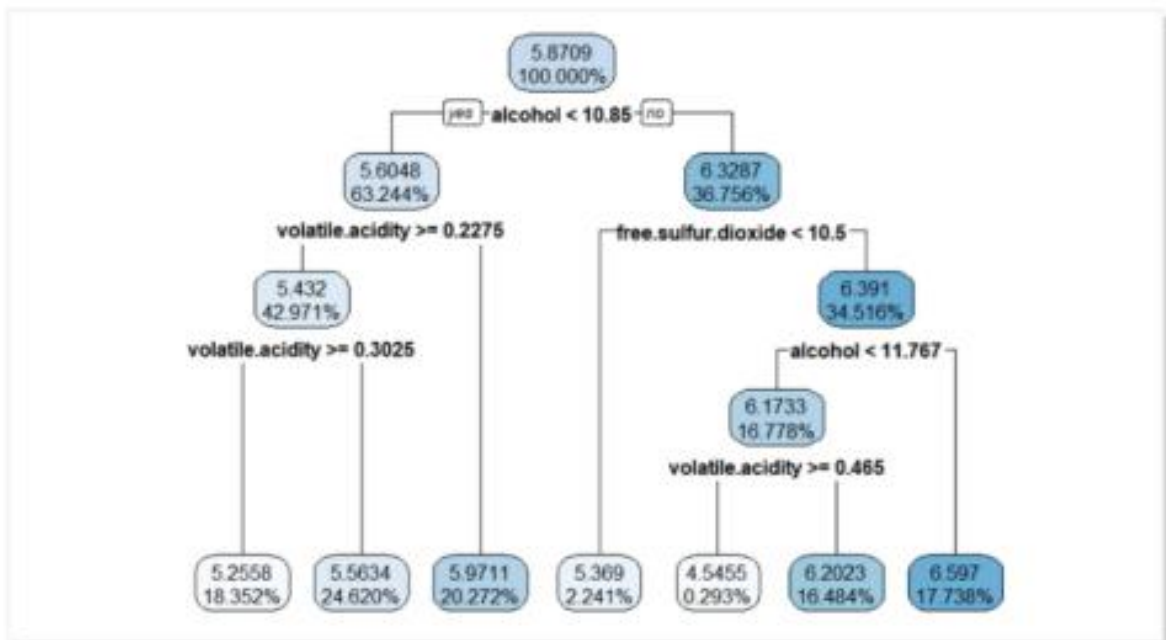
`fallen.leaves = TRUE`

바닥에 정렬

* 문서화와 모델의 시각화

* 노드는 터미널 노드 또는 잎 노드로, 노드에서 예측

◦ 도식화 함수 파라미터의 시각화



회귀 트리와 모델 트리로 품질 평가하기

2. 회귀 트리와 모델 성능 평가

1) 4단계 : 데이터 모델 성능 평가

◦ 예측 회귀 트리 모델

- * 테스트 데이터에 대한 예측에 회귀 트리 모델 사용
- * `predict()` 함수 사용
- * 결과 변수에 적합한 추정 숫자 값의 반환
- * 데이터 범위 : 예측이 실제 데이터 값보다 훨씬 좁음
- * 예측된 품질 값과 실제 품질 값 간의 상관 관계는 모델의 성능으로 측정함
- * `cor()` 함수 사용 : 예측값과 실제값의 일치 정도 측정
- * 상관 관계 0.54로 수용 가능함
- * 상관 관계는 예측과 실제값의 연관된 강도만 측정함

◦ 평균 절대 오차로 성능 평가하기

- * 모델의 성능 : 평균적으로 예측값과 실제값의 차이를 의미함
- * 측정치 : 평균 절대 오차(Mean Absolute Error)
 - 오차 절대값의 평균 계산
 - `WMAE()` 함수를 생성하여 예측된 값과 실제값의 차이(오차) 계산
 - 결과 : 와인의 품질은 좋지도 않고 나쁘지도 않음
 - 품질 점수는 대략 5~6 정도
- * 회귀 트리($WMAE = 0.59$)
 - 대체 평균($WMAE = 0.67$)보다 실제 품질 점수가 유사, 연구 필요
- * 신경망($WMAE = 0.58$)
 - 서포트 벡터 머신 $WMAE$ 0.45에 비해 개선의 여지가 있음

회귀 트리와 모델 트리로 품질 평가하기

2) 5단계 : 분석 모델 성능 개선

◦ 모델 트리 알고리즘 적용

- * 모델 트리 : 잎 노드를 회귀 모델로 대체하여 회귀 트리가 확장됨
- * 잎 노드에서 예측된 하나의 값만 사용할 때 더 정확한 결과를 가져옴
- * 모델 트리 기술로 큐비스트(Cubist) 알고리즘 적용
 - 결정 트리 구축
 - 트리의 가지 기반으로 결정 규칙 생성
 - 각 잎 노드에서 회귀 모델 구축
 - 가지치기와 부스팅 등의 추가적인 휴리스틱(heuristics)을 사용하여 예측 품질 개선
 - Cubist 패키지의 `cubist()` 함수 사용

◦ 추정된 회귀 효과

- * 와인 표본에만 적용됨
- * 각 결정 규칙은 모두 25개의 선형 모델로 구축
- * 잔여 설량과 나머지 10개 특징의 효과에 따른 파라미터 추정
- * `predict()` 함수 : 예측값의 벡터 결과 도출
- * 모델 트리 : 회귀 트리에 비해 더 넓은 범위의 값 예측
 - 상관 관계는 증가, 평균 절대 오차는 감소, 신경망 모델 성능은 증가
- * 서포트 벡터 머신 모델에 대한 평균 절대 오차값은 0.53에 근접함
- * 실행 결과 분석
 - 예측값과 참값의 평균 절대 오차(사용자 정의 함수)

```
> WMAE(whitewines_test$quality, pr.cubist)
[1] 0.5336441
```

- 모델은 평균 절대 오차를 축소시킴
- 신경망 모델의 성능을 능가함

3. 데이터 예측과 회귀 방법

회귀의 이해와 다중 선형 회귀 분석

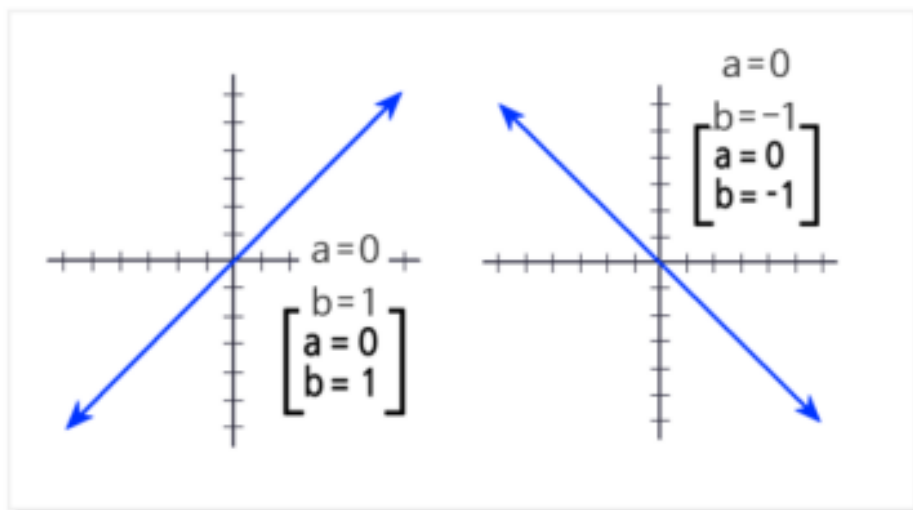
1. 회귀의 이해

1) 회귀

- 한 개의 숫자 종속 변수(dependent variable, 예측값)
- 한 개 이상의 숫자 독립 변수(independent variable)
- 예측 변수(predictors) 사이의 관계 명시
- 종속 변수는 독립 변수에 따라 달라짐
 - * 가장 단순한 회귀 형태(독립 변수와 종속 변수는 직선 관계)
 - * 기본적인 기울기 절편 형식(slope-intercept form)

$$y = a + bx$$

- 종속 변수는 독립 변수에 따라 달라짐
 - * 기울기와 절편 직선
 - y는 종속 변수, x는 독립 변수, 기울기 항 b, x가 증가
 - 직선이 어느 정도 증가하는지 명시함



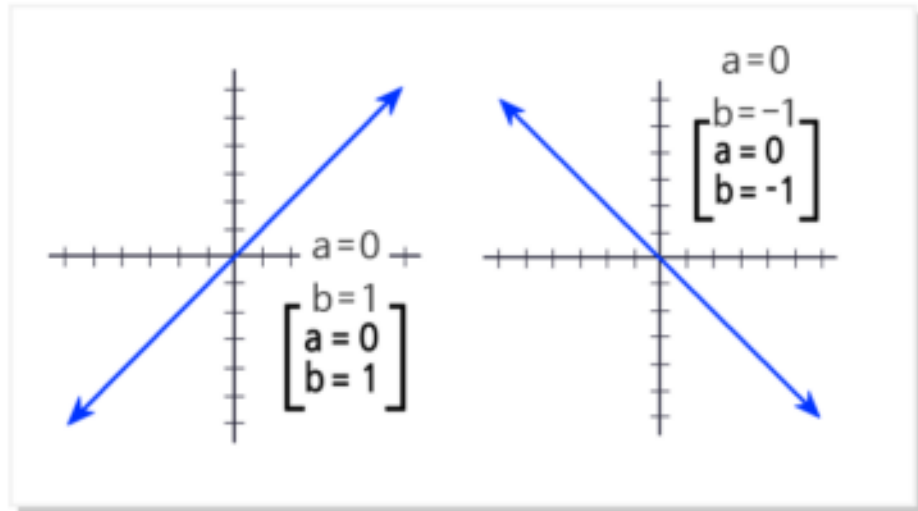
→ 양수 : 기울기가 위쪽 직선

→ 음수 : 기울기가 아래쪽 직선

회귀의 이해와 다중 선형 회귀 분석

1) 회귀

- a항 절편(intercept)



* 직선이 세로 y 축과 교차하는 지점

* $x=0$ 일 때의 y 값 표시

2) 단순 선형 회귀(Simple Linear Regression)

- 단순 선형 회귀 모델

* 하나의 종속 변수와 독립 예측 변수 간의 관계

* 방정식 형태의 직선 정의

$$y = \alpha + \beta x$$

* 절편 α (알파) : 직선이 y 축과 만나는 지점

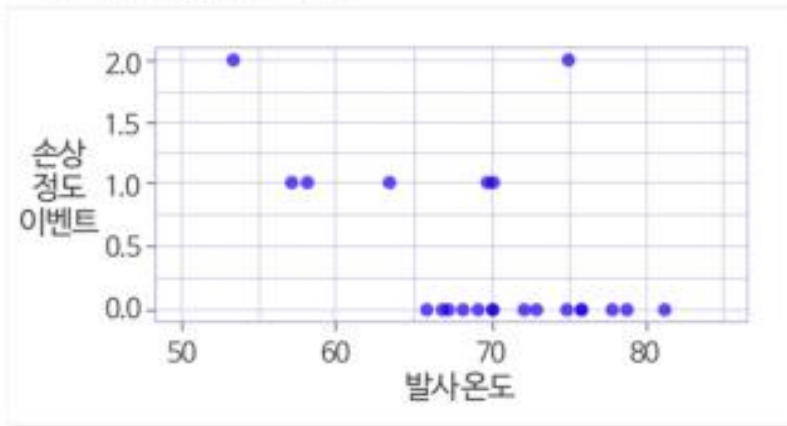
* 기울기 β (베타) : x 의 증가에 대한 y 의 변화량

회귀의 이해와 다중 선형 회귀 분석

2) 단순 선형 회귀(Simple Linear Regression)

◦ 단순 선형 회귀 모델

* 우주선 오링의 손상 및 발사 온도



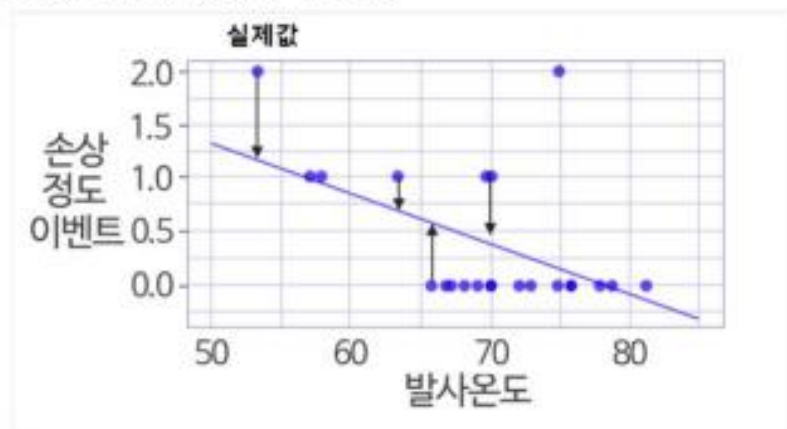
3) 보통 최소제곱 추정

◦ α, β 의 최적 추정치 결정

* 보통 최소 제곱법(Ordinary Least Squares) 추정 방법

- 기울기와 절편의 오차 제곱 합(Sum of Squared Error) 최소화
- 오차 : y 의 예측값과 실제값 간의 수직 거리 차이
- 보통 오차 : 모집단 실제값 간의 차이
- 잔차 : 표본과 실제값 간의 차이

* 우주선 오링의 손상 및 발사 온도



[손실 이벤트와 온도 간의 관계 모델링 및 회귀선]

회귀의 이해와 다중 선형 회귀 분석

4) 상관 관계

- 두 변수 간의 상관 관계(correlation)
 - * 두 변수 간의 상관 관계(correlation)
 - 변수의 관계가 직선에 가까운 정도를 표시하는 숫자
 - * 조건이 없다면 피어슨 상관 계수(Pearson's correlation coefficient)
 - * 상관 관계는 -1 ~ +1 사이의 범위
 - * 최댓값과 최솟값의 구조인 완벽한 선형 관계
 - * 0에 가까울수록 상관 관계에서 선형 관계가 없음
- 함수
 - * cov() : 공분산 함수
 - * sd() : 표준 편차 함수

회귀의 이해와 다중 선형 회귀 분석

2. 다중 선형 회귀

1) 다중 선형 회귀(Multiple Linear Regression)

- 하나 이상의 독립 변수
- 숫자 예측 작업 : 다중 선형 회귀를 사용할 가능성이 높음
- 다중 회귀 : 단순 선형 회귀의 확장된 형태
 - * 다중 회귀 방식과 단순 선형 회귀 방식
 - 두 방식의 목표는 유사함
(선형 방정식의 예측 오차를 최소화한 기울기 계수 값 검색)
 - 주요 차이점 : 추가 독립 변수를 위한 추가 항목의 유무

◦ 수식

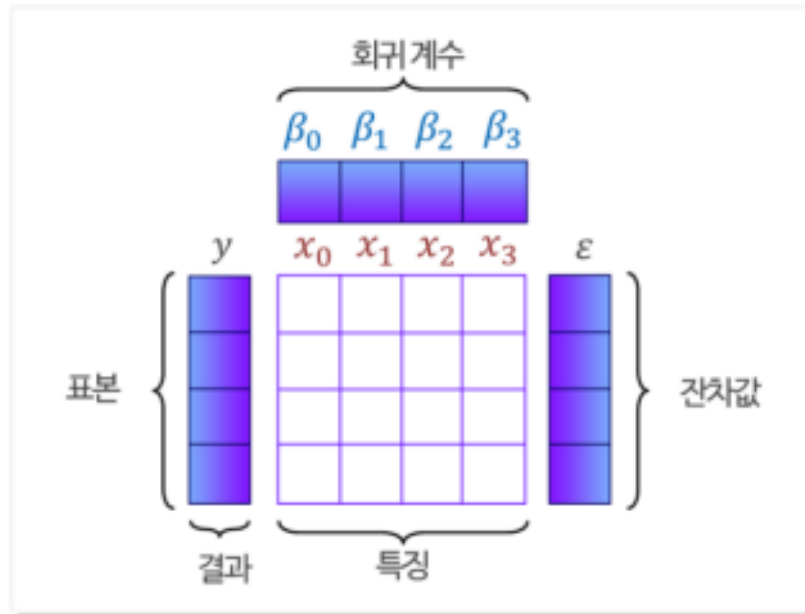
$$y = \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_i x_i + \varepsilon$$

- * 종속 변수 y
- * 절편 항목 α
- * 각 특징 i 에 대해 추정된 β 값
- * x 변수를 곱한 후 더한 값

회귀의 이해와 다중 선형 회귀 분석

2) 다중 선형 회귀의 장단점

- 다중 회귀 최소화, x 와 y 를 연계한 β 값 검색



- 장점
 - * 숫자 데이터의 모델링에 적합한 일반적인 방법
 - * 대다수 모델링 작업에 적용
 - * 특징과 결과 간의 관계 강도와 크기에 대한 추정치 제공
- 단점
 - * 데이터에 대한 강한 가정
 - * 모델 형태가 사용자에게 따라 사전에 지정
 - * 누락된 데이터는 처리 안 함
 - * 숫자 특징만 처리함
 - * 범주 데이터는 추가적인 준비 필요
 - * 모델을 이해하기 위한 통계 지식 필요

선형 회귀를 활용하여 예측하기

1. 선형 회귀 예측 준비

1) 피보험자 인구의 의료비 예측 모델

- 피보험자 인구의 의료비 예측 모델 개발
 - * 시간과 비용 절약
- 환자 데이터 이용
 - * 인구 세그먼트별 평균 의료비 예측
- 추정
 - * 예상 치료비에 따라 연간 보험료를 높게 또는 낮게 설정하는 보험 통계 생성 가능

2) 1단계 : 분석 데이터 수집

- 환자의 가상 의료비
 - * 모의 데이터 셋 활용
 - * 미국 통계국의 인구 통계 이용(실제 질병 반영)
 - * 의료보험에 등록된 1,338명의 정보 활용
 - * 개별적인 환자의 특성과 특정 연도의 의료보험으로 청구된 전체 의료비의 특징
- 변수 구성
 - * age : 주 수익자의 연령, 정수형모의 데이터 셋 활용
 - * sex : 보험 계약자 성별 - 여성, 남성으로 구분모의 데이터 셋 활용
 - * bmi : 키 기준에 따른 초과, 미달 몸무게, 신체 용적 지수
 - 몸무게 (kg)를 키(m)의 제곱으로 나눈 값, 18.5~24.9의 범위모의 데이터 셋 활용
 - * children : 의료보험의 대상 자녀 수/부양가족 수, 정수형
 - * smoker : 피보험자의 흡연 여부, 범주형 변수(예, 아니오)
 - * region : 수익자의 거주 지역, 남서, 북서, 북동, 남동으로 구분(4개)

선형 회귀를 활용하여 예측하기

3) 2단계 : 분석 데이터 탐색과 준비

◦ 특징 간 관계 탐색 : 상관 행렬

* 회귀 모델을 데이터에 접합하기 전 탐색

* 독립 변수와 종속 변수의 상관 관계에 유용함

* 상관 행렬 : 관계에 대한 신속한 요약 제공

* 변수 집합

- 상관 행렬 : 각 변수 쌍의 상관 관계 제공

- insurance_exp 데이터프레임에 4개의 숫자 변수의 상관 행렬 생성

- cor() 함수로 명령함

- age와 bmi는 양의 상관 관계

- 나이가 들수록 몸무게가 증가하는 경향

- age와 expenses, bmi와 expenses, children과 expenses 변수가 양의 상관관계

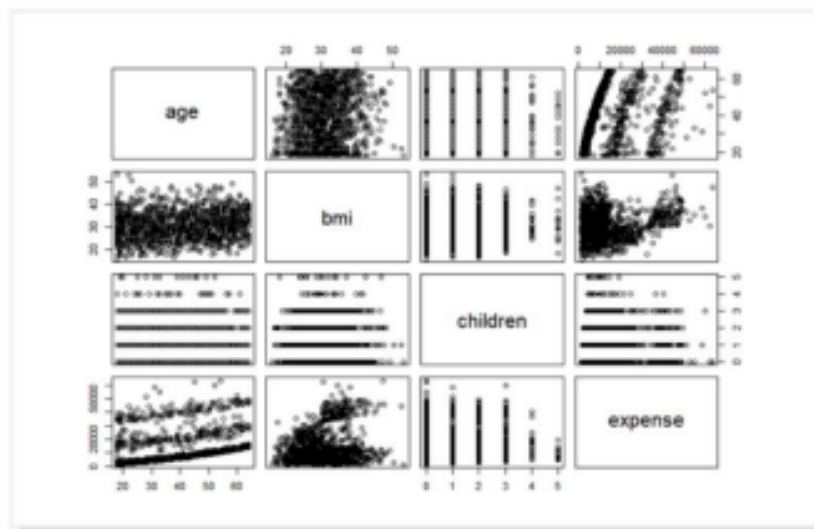
◦ 특징 간 관계 시각화 : 산포도 행렬

* 숫자 특성에 대한 산포도 행렬

- 산포도를 활용하여 특징 간의 관계를 시각화 함

- 모든 관계별로 산포도 생성

- 격자로 정렬된 그룹으로 산포도 행렬 생성



선형 회귀를 활용하여 예측하기

3) 2단계 : 분석 데이터 탐색과 준비

◦ 특징 간 관계 시각화 : 산포도 행렬

* pair() 함수 사용

* 대각선에 특징별 값 분포를 표시하는 히스토그램

* 산포도 행렬 : 세 개 이상의 변수의 패턴을 검색할 때 사용

* 타원 모양 개체 : 상관 관계 강도를 타원형으로 시각화 함

* 타원이 가로로 커질수록 강한 상관 관계 경향

* 산포도 행렬에 세부 사항 반영

- bmi, children은 아주 약한 상관 관계(타원 0.01)

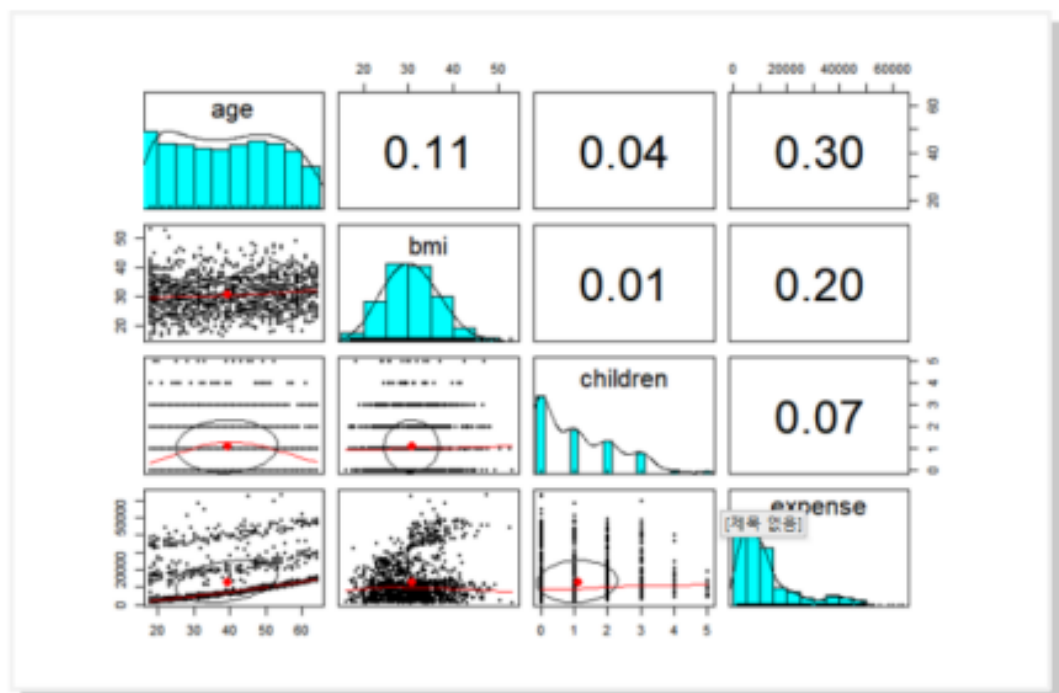
- age, expense는 강한 상관 관계(타원 0.30)

- 타원 점은 x 와 y 의 평균

- 산포도에 표시된 곡선

- age와 expense의 곡선은 u자를 뒤집은 모양이며, 중년이 최고점

- age와 bmi의 뾰족곡선은 상승 직선이고, 몸무게는 연령이 증가할수록 증가



선형 회귀를 활용하여 예측하기

3) 2단계 : 분석 데이터 탐색과 준비

- 특징 간 관계 시각화 : 산포도 행렬

- * 로스곡선(loess curve)

- 산포도에 표시된 곡선

- age와 expense의 곡선은 u자를 뒤집은 모양이며,

- 중년이 최고점

- age와 bmi의 로스곡선은 상승 직선이고, 몸무게는 연령이 증가할수록 증가함

선형 회귀를 활용하여 예측하기

2. 선형 회귀 모델 훈련

1) 3단계 : 분석 데이터 모델 훈련

◦ 선형 회귀 모델 적합

* `lm()` 함수 사용

* 독립 변수와 전체 의료비와의 상관관계
(age, children, bmi, sex, smoker, region)

* R 수식 구문의 모델 설명에서 틸드(tilde)문자(~) 사용

* 선형 회귀 모델 결과

- 노령, 흡연, 비만은 건강 문제와 연관되는 경향
- 가족 구성원이 추가에 따라, 연별 신체 검사, 의사 방문, 예방 접종 및 예방 치료 증가
- 모델의 적합성 여부는 검증되지 않음
- 나이가 많을수록 의료비 \$256.80 정도 증가
- 추가 자녀 한 명당 매해 의료비 평균 \$475.70 증가
- bmi 단위가 증가할수록 연간 의료비가 평균 \$339.30 증가

Coefficients:		
(Intercept)	age	sexmale
-11941.6	256.8	-131.4
bmi	children	smokingyes
339.3	475.7	23847.5
regionsnorthwest	regionssoutheast	regionssouthwest
-352.8	-1035.6	-959.3

선형 회귀를 활용하여 예측하기

2) 4단계 : 분석 모델 성능 평가

- p-값이 통계적으로 유의미함
 - * 모델 : 매우 유의미한 변수를 포함함
 - * 변수들 : 논리적인 방식의 결과로 연관됨
- 다중 R-제곱 값 모델
 - * 종속 변수 값의 적합성 여부 측정

Residual standard error: 6062 on 1329 degrees of freedom
Multiple R-squared: 0.7509, Adjusted R-squared: 0.7494
F-statistic: 500.9 on 8 and 1329 DF, p-value: < 2.2e-16

- * R-제곱 값 : 0.7494, 모델의 종속 변수 변화량 : 약 75%
 - 설명 변수의 개수가 다른 모델의 성능 비교에 유용함
- * 모델 예측 오류율 : 27%
 - 결론 : 초기 모델이므로 성능이 부족할 수 있음
- * 62개의 대출 채무 불이행 중 26개(42%)만 정확하게 예측됨
- * 채무 불이행으로 인한 손실 은행의 손해에 따라 크게 발생

선형 회귀를 활용하여 예측하기

3. 선형 회귀 모델 개선

1) 5단계 : 분석 모델 성능 개선

- 모델 명시 : 비선형 관계 추가
 - * 선형 회귀에서 독립 변수와 종속 변수 간의 관계는 선형이 필수는 아님
 - * 의료비 지출 연령의 영향은 전체 연령대에서 일정하지 않음
 - * 모델은 의료비에 대한 나이의 선형적 영향과 비선형적 영향으로 구분
- 변환 : 숫자 변수를 이진 변수로 전환
 - * 가정 : 특정한 임계값에 도달한 후 영향력이 있음
 - * 체중이 정상 범위인 사람은 의료비에 대한 bmi의 영향이 없음
 - * 비만인 사람의 bmi는 상대적으로 높은 의료비와 연관성이 강함
 - * bmi에서 비만의 영향 여부에 따라 개선될 모델에서 bmi 변수의 대체 또는 추가 가능
- 모델 명시 : 상호 작용 영향 추가
 - * 특징이 종속 변수에 결합되어 나타난 영향 여부를 어떻게 분석할 것인가?
 - * 비만과 흡연은 유해한 영향 요소
 - * 두 특징이 결합된 상태일 때 상호 작용함
- 모두 병합 : 향상된 회귀 모델
 - * 의료비와 환자 특성 사이의 연관성 있는 회귀 수식
 - * 나이의 비선형 항목 추가
 - * 비만의 지시 변수 생성
 - * 비만과 흡연 사이의 상호 작용 명시

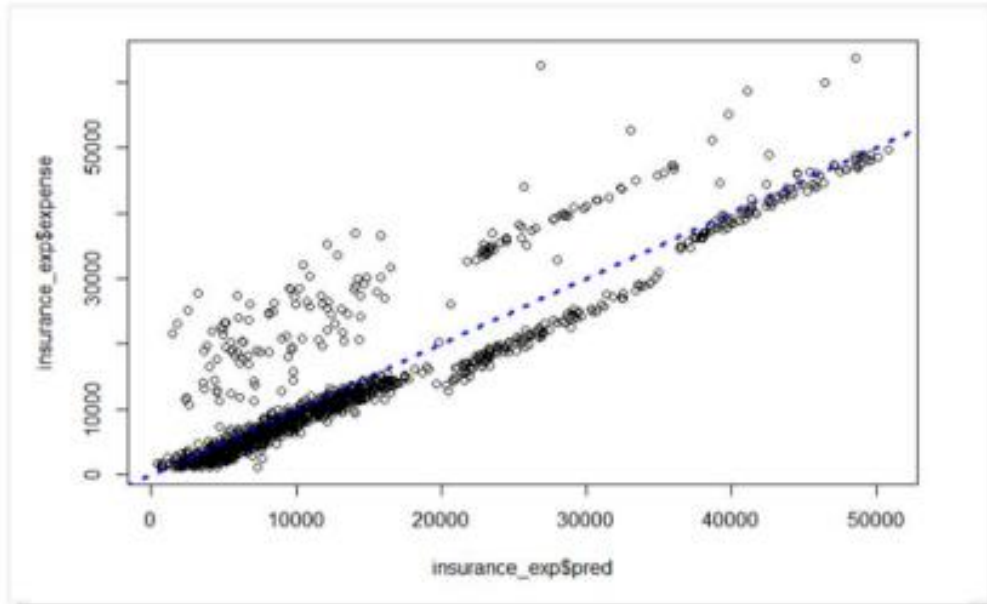
선형 회귀를 활용하여 예측하기

1) 5단계 : 분석 모델 성능 개선

- plot() 함수를 통해 상호 관계 도식화, 기울기, 절편 반영

```
plot(insurance_exp$pred, insurance_exp$expense)  
# 상호 관계 도식화, 기울기가 1, 절편이 0  
abline(a = 0, b = 1, col = "blue", lwd = 3, lty = 3)
```

- * a, b에 해당하는 항등 직선 추가
 - * col 선 색상
 - * lwd 너비
 - * lty 형식
- 직선 위쪽의 대각선으로부터 멀리 위치한 점의 분포는 실제 비용이 기댓값보다 큼
 - 선 아래는 예상 의료비보다 실제 비용이 더 적음



선형 회귀를 활용하여 예측하기

1) 5단계 : 분석 모델 성능 개선

◦ 회귀 모델 예측

- * 추정 회귀 계수와 통계량 조사
- * 회귀 모델을 활용하여 건강 보험 등록자의 비용 예측
- * 예측 프로세스 설명(predict() 함수를 사용하여 원시 훈련 데이터에 적용)
- * 건강 보험 등록자 비용 예측 실행 결과
 - 실행 결과 : 0.91로 예측과 실제값 사이에 매우 강한 선형 관계
 - 상관계수 값이 0.91로 예측과 실제값 사이의 강한 선형 관계 확인
 - 모델이 정확하다는 의미
 - 산포도도 유용함

```
> insurance_exp$pred <- predict(ins_model33,  
  insurance_exp)  
> cor(insurance_exp$pred, insurance_exp$expense)  
[1] 0.9050194
```

4. 로지스틱 회귀 모델

1. 로지스틱 회귀 모델 이해하기

1) 로지스틱 회귀란?

로지스틱 회귀는 이름에 “회귀”가 들어가지만, 실제로는 **분류 문제**에 많이 사용하는 머신러닝 알고리즘이다.

예를 들어 다음과 같은 문제에 사용할 수 있다.

- 이메일이 스팸인지 아닌지 분류
- 환자가 질병이 있는지 없는지 분류
- 학생이 합격인지 불합격인지 분류
- 붓꽃이 어떤 품종인지 분류

Iris 데이터셋에서는 붓꽃을 다음 3개 품종 중 하나로 분류한다.

- setosa
- versicolor
- virginica

즉, 이 실습의 목표는 붓꽃의 꽃받침과 꽃잎 정보를 이용하여 품종을 예측하는 것이다.

2) 로지스틱 회귀의 기본 원리

로지스틱 회귀는 입력 데이터에 가중치를 곱하고 편향을 더해서 각 클래스의 점수를 계산한다.

예를 들어 Iris 데이터는 입력 특성이 4개이다.

- sepal length
- sepal width
- petal length
- petal width

모델은 이 4개 값을 이용하여 각 품종에 대한 점수를 계산한다.

입력 데이터 4개

↓

선형 계산

↓

setosa 점수

versicolor 점수

virginica 점수

↓

가장 점수가 높은 품종 선택

PyTorch 코드에서는 이 부분을 다음 계층으로 구현한다.

```
nn.Linear(input_dim, output_dim)
```

Iris 데이터에서는 다음과 같다.

```
input_dim = 4
```

```
output_dim = 3
```

즉, 입력 4개를 받아서 출력 3개를 만드는 구조이다.

3) 이진 분류와 다중 클래스 분류

로지스틱 회귀는 크게 두 가지 방식으로 사용할 수 있다.

이진 분류

결과가 2개인 경우이다.

예:

합격 / 불합격

스팸 / 정상

질병 있음 / 질병 없음

다중 클래스 분류

결과가 3개 이상인 경우이다.

Iris 데이터셋은 3개 품종을 분류하므로 다중 클래스 분류이다.

setosa / versicolor / virginica

PyTorch에서는 다중 클래스 분류를 할 때 보통 다음 조합을 사용한다.

```
nn.Linear()
```

```
nn.CrossEntropyLoss()
```

2. Iris 데이터셋 탐색 및 준비하기

1) Iris 데이터셋이란?

Iris 데이터셋은 머신러닝 입문에서 가장 많이 사용하는 대표적인 분류 데이터셋이다.

데이터는 붓꽃의 네 가지 측정값을 가진다.

특성	의미
sepal length	꽃받침 길이
sepal width	꽃받침 너비
petal length	꽃잎 길이
petal width	꽃잎 너비

정답 라벨은 붓꽃의 품종이다.

라벨 번호	품종
0	setosa
1	versicolor
2	virginica

2) 데이터 불러오기

제공된 코드에서는 sklearn.datasets의 load_iris() 함수를 사용한다.

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

여기서 X는 입력 데이터이고, y는 정답 라벨이다.

X = 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비

y = 실제 붓꽃 품종 번호

3) 데이터프레임으로 확인하기

코드에서는 pandas DataFrame을 사용하여 데이터를 표 형태로 확인한다.

```
df = pd.DataFrame(X, columns=feature_names)
```

```
df["target"] = y
```

```
df["species"] = [target_names[label] for label in y]
```

이렇게 하면 숫자 데이터와 품종 이름을 함께 확인할 수 있다.

데이터 분석을 시작하기 전에 반드시 다음을 확인해야 한다.

- 데이터 행 개수
- 입력 특성 개수
- 정답 클래스 개수
- 클래스별 데이터 개수
- 평균, 표준편차, 최솟값, 최댓값

3. 학습 데이터와 테스트 데이터 분리

1) 왜 데이터를 나누는가?

머신러닝 모델은 학습한 데이터에서는 성능이 좋아 보일 수 있다.

하지만 중요한 것은 처음 보는 데이터도 잘 맞는가이다.

그래서 데이터를 두 부분으로 나눈다.

구분	목적
학습 데이터	모델을 훈련하는 데 사용
테스트 데이터	훈련 후 성능을 평가하는 데 사용

제공된 코드에서는 전체 데이터의 80%를 학습용, 20%를 테스트용으로 사용한다.

```
test_size=0.2
```


2) stratify=y의 의미

코드에는 다음 옵션이 사용된다.

```
stratify=y
```

이 옵션은 각 품종의 비율이 학습 데이터와 테스트 데이터에 비슷하게 유지되도록 한다.

예를 들어 setosa, versicolor, virginica가 각각 비슷한 비율로 들어가야 모델 평가가 공정해진다.

4. 데이터 표준화

1) 표준화가 필요한 이유

로지스틱 회귀는 입력 데이터의 크기 차이에 영향을 받을 수 있다.

예를 들어 어떤 특성은 값의 범위가 0~1이고, 어떤 특성은 0~100이면 모델이 큰 값을 가진 특성을 더 중요하게 볼 수 있다.

그래서 입력 데이터를 평균 0, 표준편차 1이 되도록 변환한다.

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

2) fit_transform과 transform의 차이

학습 데이터에는 다음을 사용한다.

```
fit_transform()
```

학습 데이터의 평균과 표준편차를 계산한 뒤 변환한다.

테스트 데이터에는 다음을 사용한다.

```
transform()
```

테스트 데이터는 학습 데이터에서 계산한 기준을 그대로 사용해야 한다.

테스트 데이터에 fit_transform()을 사용하면 테스트 정보가 학습 과정에 섞이는 문제가 생긴다.

이것을 데이터 누수라고 한다.

5. PyTorch Tensor 변환

1) Tensor란?

Tensor는 PyTorch에서 사용하는 기본 데이터 형식이다.

NumPy 배열이나 pandas 데이터프레임을 그대로 모델에 넣을 수 없기 때문에 Tensor로 변환해야 한다.

```
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
```

```
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
```

2) 입력 데이터와 정답 라벨의 자료형

입력 데이터는 실수형이어야 한다.

```
dtype=torch.float32
```

정답 라벨은 정수형이어야 한다.

```
dtype=torch.long
```

CrossEntropyLoss는 정답 라벨을 0, 1, 2와 같은 정수 클래스 번호로 받기 때문이다.

6. PyTorch 로지스틱 회귀 모델 만들기

1) 모델 클래스 구조

제공된 코드는 nn.Module을 상속하여 모델 클래스를 만든다.

```
class LogisticRegressionClassifier(nn.Module):
```

PyTorch에서 직접 모델을 만들 때는 보통 nn.Module을 상속한다.

2) init() 메서드

__init__()에서는 모델에 필요한 계층을 정의한다.

```
self.linear = nn.Linear(input_dim, output_dim)
```

Iris 데이터에서는 다음 의미이다.

입력 특성 4개 → 출력 클래스 3개

3) forward() 메서드

forward()는 입력 데이터가 모델 안에서 어떻게 계산되는지 정의한다.

```
logits = self.linear(x)
return logits
```

여기서 logits는 Softmax를 적용하기 전의 원시 점수이다.

CrossEntropyLoss가 내부적으로 Softmax를 처리하므로 모델 안에서 Softmax를 직접 적용하지 않는다.

7. 손실 함수와 최적화 알고리즘

1) 손실 함수

손실 함수는 모델의 예측이 실제 정답과 얼마나 다른지 계산한다.

제공된 코드에서는 다중 클래스 분류에 적합한 손실 함수인 CrossEntropyLoss를 사용한다.

```
criterion = nn.CrossEntropyLoss()
```

손실값이 작을수록 모델이 정답에 가깝게 예측하고 있다는 의미이다.

2) 최적화 알고리즘

최적화 알고리즘은 손실값을 줄이기 위해 모델의 가중치를 수정한다.

제공된 코드에서는 Adam을 사용한다.

```
optimizer = optim.Adam(model.parameters(), lr=0.05)
```

lr은 learning rate, 즉 학습률이다.

학습률이 너무 크면 학습이 불안정할 수 있고, 너무 작으면 학습이 느려질 수 있다.

8. 모델 학습 과정

모델 학습은 여러 번 반복하면서 가중치를 조금씩 수정하는 과정이다.

제공된 코드에서는 500번 반복한다.

```
epochs = 500
```

학습 과정은 다음 순서로 진행된다.

1. 모델을 학습 모드로 설정
2. 이전 기울기 초기화
3. 예측값 계산
4. 손실값 계산
5. 역전파 수행
6. 파라미터 업데이트
7. 손실값 저장

PyTorch 코드 흐름은 다음과 같다.

```
optimizer.zero_grad()
outputs = model(X_train_tensor)
loss = criterion(outputs, y_train_tensor)
loss.backward()
optimizer.step()
```

초보자는 이 순서를 반드시 기억해야 한다.

9. 모델 평가

1) 평가 모드

학습이 끝난 후에는 모델을 평가 모드로 바꾼다.

```
model.eval()
```

평가할 때는 가중치를 업데이트하지 않으므로 기울기 계산을 끈다.

```
with torch.no_grad():
```

이렇게 하면 메모리 사용량이 줄고 계산 속도가 빨라진다.

2) 예측 클래스 선택

모델은 각 클래스에 대한 점수를 출력한다.

예:

setosa 점수: 1.2

versicolor 점수: 0.5

virginica 점수: -0.7

가장 높은 점수를 가진 클래스가 최종 예측 결과가 된다.

```
_ test_predicted = torch.max(test_outputs, dim=1)
```

3) 정확도

정확도는 전체 데이터 중 모델이 맞힌 비율이다.

정확도 = 맞힌 개수 / 전체 개수

예를 들어 테스트 데이터 30개 중 29개를 맞히면 정확도는 다음과 같다.

$29 / 30 = 0.9667$

즉, 약 96.67%이다.

4) 분류 리포트

제공된 코드는 classification_report()를 사용한다.

classification_report(y_true_names, y_pred_names)

분류 리포트에서는 다음 지표를 확인할 수 있다.

지표	의미
precision	모델이 해당 클래스로 예측한 것 중 실제로 맞은 비율
recall	실제 해당 클래스 중 모델이 맞힌 비율
f1-score	precision과 recall의 균형 지표
support	실제 데이터 개수

5) 혼동 행렬

혼동 행렬은 실제 클래스와 예측 클래스를 표로 비교한 것이다.

```
confusion_matrix(y_true_names, y_pred_names)
```

혼동 행렬을 보면 어떤 품종을 어떤 품종으로 잘못 분류했는지 확인할 수 있다.

예를 들어 versicolor를 virginica로 잘못 예측했다면 두 품종의 꽃잎 길이와 너비가 비슷하기 때문일 수 있다.

10. 새 데이터 예측

학습된 모델은 새로운 붓꽃 데이터도 예측할 수 있다.

입력 순서는 다음과 같다.

```
[sepal length, sepal width, petal length, petal width]
```

새 데이터를 예측할 때도 반드시 학습 때 사용한 scaler로 표준화해야 한다.

```
new_data_scaled = scaler.transform(new_data_np)
```

그 후 Tensor로 변환하여 모델에 입력한다.

```
new_data_tensor = torch.tensor(new_data_scaled, dtype=torch.float32)
```

최종적으로 가장 높은 점수를 가진 클래스가 예측 품종이 된다.

11. 학습 손실 시각화

제공된 코드는 학습 과정에서 저장한 손실값을 그래프로 출력한다.

```
plt.plot(loss_history)
```

그래프에서 손실값이 점점 감소하면 모델이 정상적으로 학습되고 있다는 의미이다.

손실값 감소 → 모델이 정답에 가까워짐

반대로 손실값이 줄지 않거나 크게 흔들리면 다음을 확인해야 한다.

- 학습률이 너무 큰지
- epoch 수가 부족한지
- 데이터 표준화가 되었는지
- 모델 구조가 적절한지

12. 테스트 데이터 시각화

제공된 코드는 꽃잎 길이와 꽃잎 너비를 기준으로 산점도를 그린다.

petal length

petal width

Iris 데이터에서는 꽃잎 길이와 꽃잎 너비가 품종 분류에 매우 중요한 특성이다.
시각화를 통해 품종별 데이터가 어느 정도 구분되는지 확인할 수 있다.

13. 정리

로지스틱 회귀는 분류 문제를 해결하는 기본적인 머신러닝 모델이다.

Iris 데이터셋에서는 꽃받침과 꽃잎의 길이, 너비를 이용하여 붓꽃 품종을 분류한다.

PyTorch에서는 다음 흐름으로 구현한다.

데이터 불러오기

↓

학습/테스트 분리

↓

표준화

↓

Tensor 변환

↓

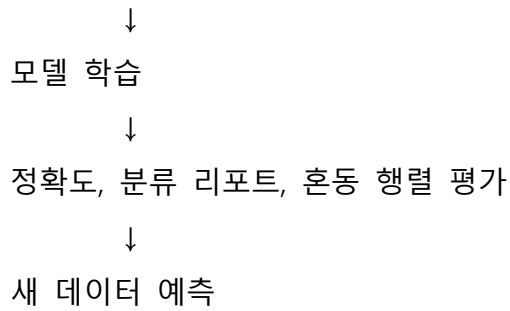
nn.Linear 모델 정의

↓

CrossEntropyLoss 설정

↓

Adam 최적화



반드시 기억해야 할 핵심은 다음과 같다.

1. 로지스틱 회귀는 분류 모델이다.
2. Iris 데이터는 4개 입력 특성으로 3개 품종을 예측한다.
3. PyTorch에서는 nn.Linear로 로지스틱 회귀를 구현한다.
4. 다중 클래스 분류에서는 CrossEntropyLoss를 사용한다.
5. 모델 평가는 정확도, 분류 리포트, 혼동 행렬로 확인한다.
6. 새 데이터를 예측할 때도 학습 데이터 기준의 표준화를 동일하게 적용해야 한다.

5. 역전파

1. 역전파란?

1) 역전파의 기본 개념

역전파는 인공신경망이 스스로 가중치를 수정하면서 학습할 수 있도록 해주는 핵심 알고리즘이다.

신경망은 처음에는 아무것도 모르는 상태에서 시작한다.

처음 가중치는 대부분 무작위로 설정된다.

그래서 처음 예측 결과는 틀릴 가능성이 높다.

예를 들어 정답이 1인데 모델이 0.3을 예측했다면 오차가 발생한다.

정답: 1

예측값: 0.3

오차: $1 - 0.3 = 0.7$

역전파는 이 오차를 이용하여 각 가중치를 어떻게 수정해야 하는지 계산한다.

즉, 역전파는 다음 질문에 답하는 과정이다.

어떤 가중치가 오차를 크게 만들었는가?

그 가중치를 얼마나 수정해야 하는가?

어느 방향으로 수정해야 오차가 줄어드는가?

2) 왜 역전파가 필요한가?

단층 퍼셉트론에서는 입력층과 출력층이 바로 연결되어 있다.

그래서 출력 오차를 보고 어떤 가중치를 수정해야 하는지 비교적 쉽게 알 수 있다.

하지만 다층 퍼셉트론은 입력층, 은닉층, 출력층으로 구성된다.

입력층 → 은닉층 → 출력층

출력층에서는 정답과 예측값을 비교할 수 있다.

하지만 은닉층은 직접 정답을 갖고 있지 않다.

따라서 은닉층의 가중치를 어떻게 수정해야 하는지 바로 알기 어렵다.

이 문제를 해결하는 방법이 역전파이다.

2. 다층 퍼셉트론 구조 이해하기

1) 다층 퍼셉트론이란?

다층 퍼셉트론은 여러 층으로 구성된 신경망이다.

기본 구조는 다음과 같다.

입력층 → 은닉층 → 출력층

각 층은 여러 개의 노드 또는 뉴런으로 구성된다.

2) 입력층

입력층은 데이터를 받아들이는 부분이다.

예를 들어 붓꽃 품종 분류라면 입력값은 다음과 같다.

꽃받침 길이

꽃받침 너비

꽃잎 길이

꽃잎 너비

3) 은닉층

은닉층은 입력값을 조합하여 중요한 특징을 찾아내는 부분이다.

은닉층은 사람이 직접 정답을 주지 않아도 내부적으로 패턴을 학습한다.

예를 들어 다음과 같은 특징을 학습할 수 있다.

꽃잎이 길면 특정 품종일 가능성이 높다.

꽃받침 너비가 크면 다른 품종일 가능성이 높다.

4) 출력층

출력층은 최종 예측 결과를 만든다.

분류 문제에서는 각 클래스에 대한 점수를 출력한다.

예를 들어 붓꽃 분류에서는 다음과 같이 출력할 수 있다.

setosa 점수: 0.9

versicolor 점수: 0.1

virginica 점수: 0.0

가장 높은 점수를 가진 클래스가 최종 예측값이 된다.

3. 기여도 할당 문제

1) 기여도 할당 문제란?

기여도 할당 문제는 신경망에서 매우 중요한 문제이다.

출력 결과가 틀렸을 때, 어느 가중치가 얼마나 책임이 있는지 판단해야 한다.

예를 들어 시험 점수가 낮게 나왔다고 가정한다.

그 원인이 다음 중 무엇인지 판단해야 한다.

공부 시간이 부족했는가?

문제를 잘못 이해했는가?

계산 실수가 있었는가?

복습을 하지 않았는가?

신경망에서도 마찬가지이다.

오차가 발생했을 때 다음을 알아야 한다.

입력층과 은닉층 사이의 가중치가 문제인가?

은닉층과 출력층 사이의 가중치가 문제인가?

어떤 가중치를 크게 수정해야 하는가?

어떤 가중치는 조금만 수정해야 하는가?

이것이 기여도 할당 문제이다.

2) 은닉층에서 문제가 어려운 이유

출력층은 정답과 직접 비교할 수 있다.

정답 - 예측값 = 오차

하지만 은닉층은 직접적인 정답이 없다.

그래서 은닉층의 각 노드가 최종 오차에 얼마나 영향을 주었는지 계산해야 한다.

역전파는 출력층에서 계산된 오차를 뒤쪽에서 앞쪽으로 전달하면서 이 문제를 해결한다.

4. 역전파의 전체 흐름

역전파는 크게 두 단계로 이루어진다.

1단계: 앞먹임 단계

2단계: 오류 후방 전파 단계

1) 앞먹임 단계

앞먹임 단계는 입력값을 신경망에 넣어 최종 예측값을 계산하는 과정이다.

입력값 입력

↓

은닉층 계산

↓

출력층 계산

↓

예측값 생성

예를 들어 입력값이 들어오면 먼저 은닉층의 값이 계산된다.

그 다음 은닉층의 출력값을 이용하여 출력층의 값이 계산된다.

2) 오류 후방 전파 단계

오류 후방 전파 단계는 예측값과 정답의 차이를 계산한 뒤, 그 오차를 출력층에서 은닉층 방향으로 거꾸로 전달하는 과정이다.

예측값과 정답 비교

↓

오차 계산

↓

출력층 가중치 수정

↓

은닉층 가중치 수정

이 과정에서 각 가중치가 오차에 얼마나 영향을 주었는지를 계산한다.

5. 앞먹임 단계 상세 설명

1) 은닉층 값 계산

입력값이 은닉층으로 전달될 때는 입력값과 가중치를 곱한 뒤 모두 더한다.

은닉층 입력값 = 입력값 × 가중치의 합

그 다음 활성화함수를 적용한다.

은닉층 출력값 = 활성화함수(은닉층 입력값)

기호로 표현하면 다음과 같다.

$$u_j = \sum v_{ji} x_i$$

$$z_j = g(u_j)$$

여기서 각 기호의 의미는 다음과 같다.

x_i : 입력값

v_{ji} : 입력층과 은닉층을 연결하는 가중치

u_j : 은닉층으로 들어가는 계산값

g : 활성화함수

z_j : 은닉층의 출력값

2) 출력층 값 계산

은닉층에서 나온 값은 출력층으로 전달된다.

출력층에서도 마찬가지로 은닉층 출력값과 가중치를 곱한 뒤 모두 더한다.

출력층 입력값 = 은닉층 출력값 × 가중치의 합

그 다음 활성화함수를 적용하여 최종 출력값을 만든다.

$$a_k = \sum w_{kj} z_j$$

$$y_k = g(a_k)$$

여기서 각 기호의 의미는 다음과 같다.

z_j : 은닉층 출력값

w_{kj} : 은닉층과 출력층을 연결하는 가중치

a_k : 출력층으로 들어가는 계산값

g : 활성화함수

y_k : 모델의 최종 예측값

6. 손실 함수 이해하기

1) 손실 함수란?

손실 함수는 모델이 얼마나 틀렸는지를 숫자로 나타내는 함수이다.

예측값이 정답과 가까우면 손실값이 작다.

예측값이 정답과 멀면 손실값이 크다.

2) 제곱 오차 손실 함수

PDF에서는 에러 제곱합 형태의 손실 함수를 설명한다.

$$E = 1/2 \sum (d_k - y_k)^2$$

여기서 각 기호의 의미는 다음과 같다.

E : 손실값

d_k : 정답값

y_k : 예측값

$d_k - y_k$: 오차

앞에 1/2을 붙이는 이유는 미분 계산을 쉽게 하기 위해서이다.

제곱을 미분하면 2가 나오는데, 앞의 1/2과 곱해져 계산이 간단해진다.

7. 경사 하강법과 가중치 수정

1) 경사 하강법이란?

경사 하강법은 손실값을 줄이는 방향으로 가중치를 조금씩 수정하는 방법이다.

쉽게 말하면 산에서 가장 낮은 곳으로 내려가는 과정과 비슷하다.

현재 위치 확인

↓

내려갈 방향 확인

↓

조금 이동

↓

다시 현재 위치 확인

↓

반복

신경망에서는 다음과 같이 동작한다.

현재 가중치 확인

↓

손실값 계산

↓

손실이 줄어드는 방향 계산

↓

가중치 수정

↓

반복

2) 가중치 업데이트 기본식

가중치는 다음 원리로 수정된다.

새 가중치 = 현재 가중치 - 학습률 × 기울기

여기서 기울기는 해당 가중치를 바꾸었을 때 손실값이 얼마나 변하는지를 의미한다.

학습률은 한 번에 얼마나 크게 수정할지를 정하는 값이다.

3) 학습률의 의미

학습률이 너무 크면 최적값을 지나칠 수 있다.

학습률 너무 큼 → 학습 불안정

학습률이 너무 작으면 학습이 너무 느리다.

학습률 너무 작음 → 학습 속도 느림

따라서 적절한 학습률을 선택해야 한다.

8. 체인룰 이해하기

1) 체인룰이란?

체인룰은 여러 함수가 연결되어 있을 때 미분을 계산하는 방법이다.

신경망은 여러 계산이 연결된 구조이다.

입력값 → 가중합 → 활성화함수 → 출력값 → 손실값

가중치가 손실값에 미치는 영향을 알기 위해서는 중간 계산들을 따라가며 미분해야 한다.
이때 체인룰을 사용한다.

2) 왜 체인룰이 필요한가?

가중치가 손실값에 직접 연결되어 있지 않기 때문이다.

가중치는 먼저 노드의 입력값을 바꾸고, 노드의 입력값은 활성화함수 결과를 바꾸고, 활성화함수 결과는 최종 예측값을 바꾸고, 예측값은 손실값을 바꾼다.

즉 다음과 같은 흐름이다.

가중치 변화



노드 입력값 변화



노드 출력값 변화



예측값 변화



손실값 변화

따라서 하나씩 연결해서 계산해야 한다.

9. 출력층 가중치 업데이트

1) 출력층의 오차 계산

출력층은 정답과 직접 비교할 수 있다.

그래서 출력층의 오차는 비교적 쉽게 계산된다.

출력층 오차 = 정답값 - 예측값

하지만 실제 가중치 수정에는 활성화함수의 미분값도 포함된다.

$$\Delta_i = (d_i - y_i) \times g'(a_i)$$

여기서 각 기호의 의미는 다음과 같다.

Δ_i : 출력층 유닛의 오차 신호

d_i : 정답값

y_i : 예측값

$g'(a_i)$: 활성화함수의 미분값

2) 출력층 가중치 수정

출력층 가중치는 다음 요소에 의해 수정된다.

출력층 가중치 변화량

= 학습률 $\times$ 출력층 오차 신호 $\times$ 은닉층 출력값

기호로 표현하면 다음과 같다.

$$w_{ij}(t+1) - w_{ij}(t) = \eta \Delta_i(t) z_j(t)$$

여기서 각 기호의 의미는 다음과 같다.

w_{ij} : 은닉층과 출력층 사이의 가중치

η : 학습률

Δ_i : 출력층의 오차 신호

z_j : 은닉층의 출력값

즉, 출력층 가중치는 다음 세 가지에 의해 바뀐다.

오차가 클수록 많이 바뀐다.

학습률이 클수록 많이 바뀐다.

입력으로 들어온 은닉층 값이 클수록 많이 바뀐다.

10. 은닉층 가중치 업데이트

1) 은닉층은 직접 오차를 알 수 없다

은닉층은 정답과 직접 비교할 수 없다.

따라서 출력층에서 전달된 오차를 이용해야 한다.

출력층의 오차가 은닉층으로 거꾸로 전달된다.

이 때문에 이름이 역전파이다.

2) 은닉층의 오차 신호

은닉층의 오차 신호는 다음 요소를 고려한다.

활성함수의 미분값

상위 출력층의 오차 신호

은닉층과 출력층 사이의 가중치

기호로 표현하면 다음과 같다.

$$\delta_i = g'(u_i) \times \sum(w_{ji} \Delta_j)$$

여기서 각 기호의 의미는 다음과 같다.

δ_i : 은닉층 유닛의 오차 신호

$g'(u_i)$: 은닉층 활성화함수의 미분값

w_{ji} : 은닉층과 출력층 사이의 가중치

Δ_j : 출력층의 오차 신호

즉, 은닉층의 책임은 출력층 오차를 보고 계산한다.

3) 은닉층 가중치 수정

은닉층 가중치는 다음 식으로 수정된다.

$$v_{ij}(t+1) - v_{ij}(t) = \eta \delta_i(t) x_j(t)$$

여기서 각 기호의 의미는 다음과 같다.

v_{ij} : 입력층과 은닉층 사이의 가중치

η : 학습률

δ_i : 은닉층의 오차 신호

x_j : 입력값

즉, 은닉층 가중치는 다음 세 가지에 의해 바뀐다.

학습률

은닉층의 오차 신호

입력값의 크기

11. 활성화함수의 미분값

1) 활성화함수란?

활성함수는 신경망이 단순한 직선 계산만 하지 않고 복잡한 패턴을 학습할 수 있도록 도와주는

함수이다.

대표적인 활성화함수는 다음과 같다.

시그모이드 함수

ReLU 함수

tanh 함수

2) 왜 미분값이 중요한가?

가중치 업데이트 식에는 활성화함수의 미분값이 포함된다.

출력층 오차 신호는 다음과 같다.

$$\Delta_i = (d_i - y_i) \times g'(a_i)$$

은닉층 오차 신호는 다음과 같다.

$$\delta_i = g'(u_i) \times \sum(w_{ji} \Delta_j)$$

두 식 모두 활성화함수의 미분값을 포함한다.

즉, 활성화함수의 미분값이 작으면 가중치 변화도 작아진다.

3) 시그모이드 함수의 특징

시그모이드 함수는 입력값을 0과 1 사이로 바꾼다.

입력값이 매우 작음 $\rightarrow$ 0에 가까움

입력값이 0 근처 $\rightarrow$ 0.5 근처

입력값이 매우 큼 $\rightarrow$ 1에 가까움

시그모이드 함수는 확률처럼 해석하기 쉬운 장점이 있다.

4) 시그모이드 함수의 미분값

시그모이드 함수의 미분값은 출력값을 이용해 간단히 표현할 수 있다.

$$g'(a) = g(a)(1 - g(a))$$

또는 출력값 y 를 사용하면 다음과 같다.

$$g'(a) = y(1 - y)$$

시그모이드 함수는 입력값이 0 근처일 때 미분값이 크고, 입력값이 너무 크거나 너무 작으면 미

분값이 0에 가까워진다.

12. 활성화함수 미분값과 학습의 관계

1) 미분값이 클 때

활성함수 미분값이 크면 가중치가 비교적 크게 수정된다.

미분값 큼



오차 신호 잘 전달



가중치 학습 잘 됨

2) 미분값이 작을 때

활성함수 미분값이 작으면 가중치 변화량도 작아진다.

미분값 작음



오차 신호 약함



가중치가 거의 수정되지 않음



학습이 느려짐

3) 기울기 소실 문제

시그모이드 함수는 입력값이 너무 크거나 너무 작으면 미분값이 거의 0이 된다.

그러면 뒤에서 전달되는 오차가 앞쪽 층으로 갈수록 매우 작아진다.

이 문제를 기울기 소실 문제라고 한다.

출력층 오차 발생



은닉층으로 전달



미분값이 작아짐



앞쪽 층으로 갈수록 오차 신호 약화



초기 층 가중치가 거의 학습되지 않음

딥러닝 초기에 깊은 신경망 학습이 어려웠던 주요 이유 중 하나가 바로 이 문제이다.

13. 역전파 학습 과정 전체 정리

역전파 학습은 다음 순서로 진행된다.

1. 입력 데이터를 신경망에 넣는다.
2. 은닉층 값을 계산한다.
3. 출력층 값을 계산한다.
4. 예측값과 정답을 비교하여 손실값을 계산한다.
5. 출력층의 오차 신호를 계산한다.
6. 출력층 가중치를 수정한다.
7. 출력층 오차를 은닉층으로 전달한다.
8. 은닉층의 오차 신호를 계산한다.
9. 은닉층 가중치를 수정한다.
10. 이 과정을 여러 번 반복한다.

14. PyTorch 관점에서 보는 역전파

PyTorch에서는 역전파를 직접 수식으로 모두 구현하지 않아도 된다.

PyTorch가 자동 미분 기능을 제공하기 때문이다.

일반적인 학습 코드는 다음과 같은 흐름을 가진다.

```
optimizer.zero_grad()
outputs = model(X_train)
loss = criterion(outputs, y_train)
loss.backward()
optimizer.step()
```

각 코드의 의미는 다음과 같다.

```
optimizer.zero_grad()
```

이전 학습에서 계산된 기울기를 초기화한다.

```
outputs = model(X_train)
```

앞먹임 단계를 수행하여 예측값을 계산한다.

```
loss = criterion(outputs, y_train)
```

예측값과 정답을 비교하여 손실값을 계산한다.

```
loss.backward()
```

역전파를 수행하여 각 가중치의 기울기를 계산한다.

```
optimizer.step()
```

계산된 기울기를 이용하여 가중치를 업데이트한다.

초보자는 PyTorch 학습 코드에서 `loss.backward()`가 역전파를 수행하는 핵심 명령이라는 점을 기억해야 한다.

15. 예제로 이해하는 역전파

간단한 예를 생각해보자.

정답값: 1

예측값: 0.2

오차: 0.8

모델의 예측값이 정답보다 너무 작다.

그러면 역전파는 예측값이 커지는 방향으로 관련 가중치를 수정한다.

반대로 다음과 같은 경우를 생각해보자.

정답값: 0

예측값: 0.9

오차: -0.9

모델의 예측값이 너무 크다.

그러면 역전파는 예측값이 작아지는 방향으로 관련 가중치를 수정한다.

즉, 역전파는 단순히 "틀렸다"만 알려주는 것이 아니라, "어느 방향으로 얼마나 고쳐야 하는지"를 알려준다.

16. 자주 헷갈리는 부분

1) 역전파는 데이터를 거꾸로 넣는 것이 아니다

역전파는 입력 데이터를 거꾸로 넣는 과정이 아니다.

입력 데이터는 앞먹임 단계에서 정상 방향으로 들어간다.

역전파에서 거꾸로 이동하는 것은 오차 정보이다.

입력 데이터: 입력층 → 은닉층 → 출력층

오차 정보: 출력층 → 은닉층 → 입력층 방향

2) 가중치는 한 번에 완성되지 않는다

가중치는 한 번 계산해서 끝나는 것이 아니다.

많은 epoch 동안 조금씩 수정된다.

1 epoch

2 epoch

3 epoch

...

반복 학습을 통해 손실값이 점점 줄어든다.

3) 모든 가중치가 같은 크기로 수정되지 않는다

각 가중치마다 오차에 기여한 정도가 다르다.

따라서 수정되는 크기도 다르다.

큰 책임이 있는 가중치는 많이 수정되고, 작은 책임이 있는 가중치는 조금 수정된다.

17. 학습정리

역전파는 다층 퍼셉트론을 학습시키기 위한 핵심 알고리즘이다.

다층 퍼셉트론에서는 출력층에서만 직접 오차를 계산할 수 있기 때문에, 은닉층의 가중치를 어떻게 수정해야 하는지가 문제가 된다.

역전파는 출력층에서 계산된 오차를 은닉층 방향으로 전달하면서 각 가중치의 책임을 계산한다.

역전파는 다음 두 단계로 구성된다.

앞먹임 단계:

입력값을 사용하여 최종 예측값을 계산한다.

오류 후방 전파 단계:

예측값과 정답의 차이를 계산하고, 그 오차를 뒤에서 앞으로 전달하여 가중치를 수정한다.

가중치 업데이트에는 다음 요소가 영향을 준다.

- 학습률
- 오차의 크기
- 입력값의 크기
- 활성화함수의 미분값
- 상위층에서 전달된 오차 신호

활성함수의 미분값은 학습에 매우 중요하다.

미분값이 크면 오차 신호가 잘 전달되고, 미분값이 너무 작으면 학습이 잘 되지 않는다.

특히 시그모이드 함수는 입력값이 너무 크거나 작을 때 미분값이 0에 가까워져 기울기 소실 문제가 발생할 수 있다.

따라서 역전파를 이해하려면 다음 핵심을 기억해야 한다.

1. 앞먹임은 예측값을 계산하는 과정이다.
2. 역전파는 오차를 뒤로 전달하는 과정이다.
3. 체인룰은 각 가중치의 책임을 계산하는 도구이다.
4. 가중치는 손실값이 줄어드는 방향으로 수정된다.
5. 활성화함수의 미분값이 작으면 학습이 느려질 수 있다.